

Performance Testing

Date	30 September 2026
Team ID	SWTID-2026-9716
Project Name	Weather-Wise
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the system behaves under expected and peak load conditions. The Weather-Wise backend was checked through API requests during development. Formal load-test measurements are not available in the provided project material, so actual performance values are marked as Not Measured rather than being invented.

Step 1: Testing Overview

Testing Tool Used	Thunder Client (API testing)
Type of Testing	API / Functional Response Testing
Target Module / API	Registration, Login, Weather API, AI Recommendation and AI Summary APIs
Test Environment	Local development environment
Test Date	SWTID-2026-7291

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Test user registration API with valid user details.	Not measured	Not measured	User should be registered successfully.
2	Test login API with valid registered credentials.	Not measured	Not measured	JWT token should be returned successfully.
3	Test weather API using a city such as Chennai.	Not measured	Not measured	Weather details should be returned correctly.
4	Test AI recommendation and weather summary APIs.	Not measured	Not measured	AI response should be returned successfully.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	Not measured	Not Tested	Formal performance measurement was not provided.
2	Response Time (Max)	< 5 seconds	Not measured	Not Tested	Formal performance measurement was not provided.
3	Throughput (Req/sec)	To be defined	Not measured	Not Tested	Load-testing results are not available.
4	Error Rate	< 1%	Not measured	Not Tested	No formal load-test result was provided.
5	CPU Utilization	< 80%	Not measured	Not Tested	Requires system monitoring during load testing.
6	Memory Utilization	< 80%	Not measured	Not Tested	Requires system monitoring during load testing.

Step 4: Observations & Analysis

Key Findings	The backend APIs were tested during development using Thunder Client. The provided material does not contain formal load-testing measurements.
Bottlenecks Identified	No specific performance bottleneck can be confirmed without formal load or stress testing.
Optimization Steps Taken	The backend is separated into routes, controllers, middleware and services to keep API processing organized. Further performance optimization should be based on actual load-test results.

Step 5: Screenshots / Evidence

Attach screenshots of Thunder Client API responses or a formal performance-testing report here. No performance-testing screenshots were provided with the source document.